INSY – Lernguide zum 1. Test

Alles aus deinem INSY-Ordner (Skripten 01–03b, Übungen 1–4.1) und den Tests 2024 + 2025. Lösungen sind eingeklappt – erst selbst rechnen, dann aufklappen.

0 · Was du können musst

Aus den alten Tests (Punkte in %):

Aufgabentyp	2024	2025	Kapitel
EER-Diagramm aus Text ergänzen (min-max + Schlüssel)	26 %	–	1
Teorey/Chen ↔ (min,max) umrechnen	6 %	–	1
Workbench: welcher Beziehungstyp (IST-Beziehung)	4 %	–	6
Schlüssel + 2NF/3NF bestimmen	8 %	15 %	3, 4
Zerlegung in 3NF (Synthese)	16 %	10 %	5
FDs aus Datentabelle prüfen	8 %	?	3
EER → Relationen in 3NF	32 %	?	2
SQL-DDL: CREATE / ALTER TABLE	–	25 %	6

Checkliste

☐ Entity, Weak Entity, Schlüssel, Beziehungen, (min,max) lesen und zeichnen
☐ Teorey ↔ (min,max) ohne Nachdenken umrechnen
☐ Generalisierung (disjunkt/überlappend, total/partiell) + Umsetzung
☐ 1:1 / 1:n / n:m / Weak Entity / IS-A / ternär → Relationen
☐ Attributhülle X^+ berechnen → alle Schlüssel finden
☐ 2NF / 3NF prüfen (prim/nichtprim!)
☐ Kanonische Überdeckung + Synthesealgorithmus
☐ FD aus Datentabelle prüfen
☐ CREATE TABLE mit PK, FK, CHECK, NOT NULL, UNIQUE, DEFAULT, AUTO_INCREMENT
☐ ALTER TABLE ADD / DROP / MODIFY, DROP TABLE

1 · EER-Modellierung

1.1 Grundbegriffe

Begriff	Symbol	Bedeutung
Entity	Rechteck	Objekttyp mit gleichen Eigenschaften (Kunde, Wohnung)
Weak Entity	Doppel-Rechteck	nur zusammen mit einer anderen Entity eindeutig (Zimmer nur mit Wohnungs-Nr.)
Identifizierende Beziehung	Doppel-Raute	verbindet Weak Entity mit ihrem „Besitzer“
Schlüsselattribut	unterstrichen	identifiziert eindeutig; bei Weak Entity: Teilschlüssel
Beschreibendes Attribut	Ellipse / Liste	normale Eigenschaft
Beziehung	Raute	Verbindung zwischen Entities, kann eigene Attribute haben (z. B. *Miete*, *seit*)

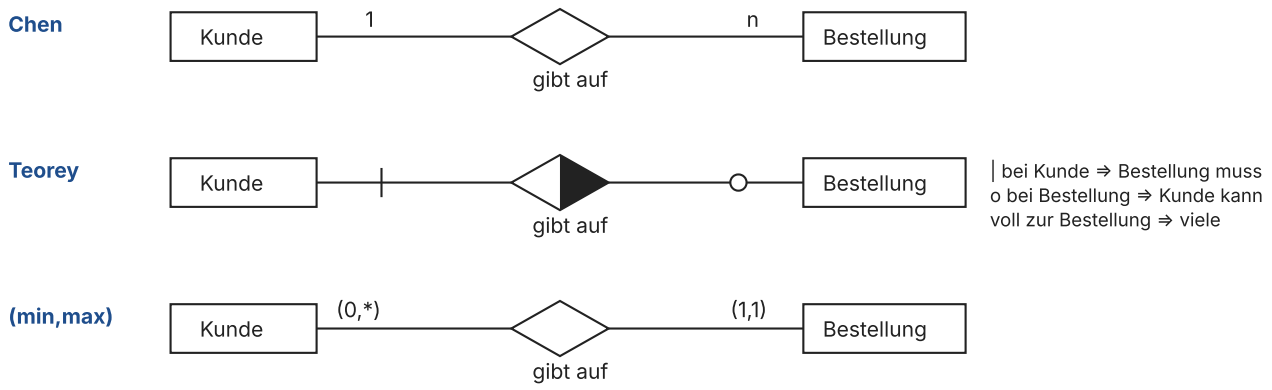
Text → EER: Signalwörter
„eindeutig durch …“ → Schlüssel
„eindeutig durch Kombination aus X von A und eigener Nr.“ → **Weak Entity** + identifizierende Beziehung
„es gibt zwei Typen / Arten von …“ → **Generalisierung** (IS-A)
„es wird gespeichert, welcher X in welchem Y bei welchem Z …“ → **ternäre Beziehung**
„… wird für die Beziehung gespeichert (Punkte, seit, Miete)“ → **Beziehungsattribut**
„mehrere Vorgesetzte / Plagiat anderer Abgaben“ → **rekursive Beziehung** (Entity mit sich selbst)
„optional“ → min = 0 · „jede/r muss“ → min = 1 · „maximal drei“ → max = 3

1.2 Komplexität

Typ	Bedeutung	Beispiel
1:1	jedes A zu genau einem B und umgekehrt	Abteilung – Leiter
1:n	A zu vielen B, jedes B nur zu einem A	Kunde – Bestellung
n:m	A zu vielen B und B zu vielen A	Student – Prüfung

1.3 Die drei Notationen – und wie man umrechnet

Beispiel: **Ein Kunde gibt beliebig viele Bestellungen auf, jede Bestellung gehört zu genau einem Kunden.**



Die wichtigste Regel
(min,max) steht beim **eigenen** Entity: „Wie oft nimmt *ein* X an der Beziehung teil?“
Chen & Teorey stehen „über Kreuz“: das Symbol bei X beschreibt das **Gegenüber Y**.

Teorey-Symbol beim Entity X	bedeutet in (min,max) für das **andere** Entity Y
| Strich	min(Y) = 1 (Y muss teilnehmen)
o Kreis	min(Y) = 0 (optional)
Rauten-Hälfte zu X **ausgefüllt**	max(Y) = * (Y hat viele X)
Rauten-Hälfte zu X **leer**	max(Y) = 1
Raute ganz voll / ganz leer	n:m / 1:1
Chen: „n“ bei X	max(Y) = *

Geprüft an Lösung Übung 1A (beide Darstellungen) und Test 2024 Aufgabe 2.

Mini-Check

Teorey: `Abteilung –o– ◇ –|– Mitarbeiter` (Raute leer, „leitet“). Wie lautet (min,max)?

▼ Lösung

o bei Abteilung ⇒ min(Mitarbeiter)=0 · | bei Mitarbeiter ⇒ min(Abteilung)=1 · Raute leer ⇒ beide max 1
Abteilung (1,1) — leitet — (0,1) Mitarbeiter → jede Abteilung hat genau einen Leiter, ein Mitarbeiter leitet max. eine.

1.4 Generalisierung (IS-A)

Subtypen erben Schlüssel + Attribute vom Supertyp. Pfeil/Dreieck zeigt zum Supertyp.

Eigenschaft	Varianten	Symbol (alternativ)
Disjunktheit	**disjunkt**: ein Objekt in max. einem Subtyp (Widerstand *oder* Kondensator) **überlappend**: in mehreren (Person ist Kunde *und* Mitarbeiter)	Kreis mit „d“ / „o“, bzw. Sechseck mit Typ-Attribut
Vollständigkeit	**total**: jedes Objekt ist in einem Subtyp **partiell**: nicht unbedingt	Doppellinie = total

Skript-Übung: überlappende Generalisierung als Beziehungen

Person ← Kunde, Person ← Mitarbeiter mit Beziehungen statt Pfeilen zeichnen.

▼ Lösung

Zwei 1:1-Beziehungen: `Person (0,1) – ist – (1,1) Kunde` und `Person (0,1) – ist – (1,1) Mitarbeiter`.
Jeder Kunde/Mitarbeiter ist **genau eine** Person (zwingend), eine Person **kann** Kunde sein (optional). Da beide Beziehungen unabhängig sind, darf eine Person beides sein → überlappend.

1.5 Ternäre & rekursive Beziehungen

Ternär: eine Raute mit 3 Entities (z. B. Mitarbeiter – Projekt – Aufgabenbereich „hat_in“, Kundenbetreuer – Kunde – Thema „berät“).
Rekursiv: Entity mit sich selbst, Rollen benennen (Vorgesetzter / Untergebener).

---

2 · EER → Relationenmodell

Schreibweise: `Relation(Schlüssel, Attribut, #Fremdschlüssel)`. Am Ende jede Relation auf 3NF prüfen.

Konstrukt	Regel	Ergebnis
Entity	eigene Relation	`A(a, …)`
1:1	Schlüssel der einen Seite in die andere. Bei optionaler Seite: FK auf die Seite, wo es **optional** ist (laut Skript), damit klar wird, welche Seite NULL haben darf	`A(a, …)` `B(b, …, #a)`
1:n	Schlüssel der **1-Seite** in die **n-Seite**	`A(a, …)` `B(b, …, #a)`
n:m	neue Relation mit beiden Schlüsseln (+ Beziehungsattribute)	`X(#a, #b, attr)`
Weak Entity	Schlüssel des Besitzers + Teilschlüssel = Schlüssel	`Zimmer(#WNr, ZNr, …)`
Generalisierung	Supertyp eigene Relation; Subtyp: **gleicher Schlüssel** (= FK auf Supertyp) + eigene Attribute. Disjunkt: oft Typ-Attribut im Supertyp	`Person(pnr, name)` `Kunde(#pnr, rabatt)`
Rekursiv 1:n	FK auf sich selbst mit Rollennamen	`Mitarbeiter(PNR, …, #VorgesetztPNR)`
Rekursiv n:m	neue Relation, Schlüssel 2× mit Rollennamen	`vorgesetzt(#MNR, #VorgesMNR)`
Ternär	neue Relation mit allen 3 Schlüsseln, Schlüssel s. u.	`ABC(a,b,c)`

> **NOT NULL (NN)**
> FK ist **NN**, wenn das Entity **zwingend** teilnimmt (min = 1). Beispiel: Bestellung (1,1) → `Bestellung(bnr, …, #knr NN)`.

2.1 Ternäre Beziehung – Schlüssel

Wie viele „1"-Seiten (Chen) gibt es?

Kardinalität	Schlüssel von ABC(a,b,c)
n : m : k (keine 1)	abc (1 Schlüssel)
eine 1-Seite (bei C)	ab – die Schlüssel der **n-Seiten**
zwei 1-Seiten	2 Schlüssel (z. B. ab, bc) → 1× PRIMARY KEY, 1× UNIQUE
drei 1-Seiten	3 Schlüssel (ab, bc, ac) → 1× PK, 2× UNIQUE

```
Faustregel: Ist C eine 1-Seite, gilt  AB → C, also braucht der Schlüssel
C nicht.
```

2.2 Vorgehen im Test

1. Jede Entity → Relation (Weak Entity: Besitzer-Schlüssel dazu).
2. Subtypen: Schlüssel vom Supertyp übernehmen.
3. 1:1 und 1:n → FK, **keine** eigene Relation.
4. n:m, ternär → eigene Relation, Beziehungsattribute dorthin.
5. Schlüssel unterstreichen, FKs mit # oder Punkt-Notation (`Versteck.vnr`).

---

3 · Funktionale Abhängigkeiten (FD)

```
X → Y  ⇔  gleiche X-Werte ⇒ gleiche Y-Werte
```

Armstrong-Regeln

```
Reflexiv:    Y ⊆ X ⇒ X → Y
Erweiterung: X → Y ⇒ XZ → YZ
Transitiv:   X → Y, Y → Z ⇒ X → Z
Vereinigung: X → Y, X → Z ⇔ X → YZ  (Zerlegung)
```

Attributhülle X^+

```
X⁺ := X
wiederhole: für jede A → B in F mit A ⊆ X⁺:  X⁺ := X⁺ ∪ B
bis sich nichts mehr ändert
```

Beispiel: R = ABCDE, F = {A → CD, BC → E}

```
A⁺  = A → ACD                          ≠ R
AB⁺ = AB → ABCD (A→CD) → ABCDE (BC→E) = R  ⇒ AB ist Schlüssel
```

3.1 Alle Schlüssel finden

> **Trick (spart 90 % Arbeit)**
> **L** = Attribute, die **nie rechts** vorkommen → in **jedem** Schlüssel.
> **R** = Attribute, die **nur rechts** vorkommen → in **keinem** Schlüssel.
> 1. L^+ berechnen. Ist $L^+ = R$ → L ist der **einzige** Schlüssel.
> 2. Sonst L um die „Mitte"-Attribute (links *und* rechts) einzeln, dann paarweise erweitern.
> Schlüssel = minimal (kein echter Teil ist auch Schlüssel).

Beispiel: R = ABCD, F = {A → BC, BC → D, D → A}

```
nie rechts: –        nur rechts: –        Mitte: A,B,C,D
A⁺ = ABCD ✓   D⁺ = DABC ✓   B⁺ = B   C⁺ = C   BC⁺ = BCDA ✓
⇒ Schlüssel: A, D, BC
```

3.2 FD aus einer Datentabelle prüfen

Für `X → Y`: suche Zeilen mit **gleichem X**. Haben sie verschiedene Y → **falsch**. Sonst **richtig** (für diese Daten).

A	B	C	D	E
a	b	c	d	e
f	b	h	d	j
k	l	m	n	o
p	q	r	s	o

```
AB → C : alle AB verschieden          ⇒ richtig
D → E  : d→e, d→j                     ⇒ falsch
D → D  : trivial                      ⇒ richtig
D → B  : d→b,b · n→l · s→q            ⇒ richtig
```

> Kommt X nur einmal vor oder ist X eindeutig ⇒ X → alles ist **richtig**. Trivial (Y ⊆ X) ⇒ immer richtig.

---

4 · Normalformen

Ziel: keine Redundanz, keine **Anomalien**:

- **Update**: gleiche Info mehrfach → beim Ändern inkonsistent („That Kind“ vs „Not That Kind“).
- **Insert**: Info kann nicht ohne andere gespeichert werden (Raum ohne Gerät).
- **Delete**: Löschen entfernt ungewollt andere Info (letztes Gerät weg → Raumbezeichnung weg).

```
prim     = in mindestens einem Schlüssel enthalten
nichtprim = in keinem Schlüssel
```

NF	Bedingung	Verletzt durch
1NF	alle Werte atomar	Liste in einer Zelle („Titelliste“), Name = Album + Interpret
2NF	1NF + jedes **nichtprime** Attribut hängt vom **ganzen** Schlüssel ab	X → A mit X ⊊ Schlüssel, A nichtprim
3NF	2NF + kein nichtprimes Attribut **transitiv** vom Schlüssel abhängig	X → A mit X kein Superschlüssel, A nichtprim
BCNF	für jede nicht-triviale X → A ist X Superschlüssel	auch wenn A prim ist

```
BCNF ⇒ 3NF ⇒ 2NF ⇒ 1NF
```

Prüf-Rezept (für jede FD X → A, A ∉ X)
1. Ist X Superschlüssel? → ok.
2. Ist A prim? → ok für 2NF und 3NF.
3. Ist X **echter Teil** eines Schlüssels? → **2NF verletzt** (also auch 3NF).
4. Sonst (X ist kein Schlüsselteil) → **3NF verletzt**, 2NF ok.

Schnell: Alle Schlüssel einelementig ⇒ automatisch 2NF. Alle Attribute prim ⇒ automatisch 3NF.

Durchgerechnete Beispiele

R, F	Schlüssel	2NF	3NF	Warum
`ABCDEF, AC→BEF`	ACD	**nein**	**nein**	AC ⊊ ACD bestimmt B,E,F
`ABCD, AB→C, C→D`	AB	**ja**	**nein**	AB→C→D transitiv
`ABCD, A→D,B→A,B→C,C→D,A→B`	A, B	**ja**	**nein**	Schlüssel einelementig; C→D, C nicht Schlüssel
`ABC, A→B, B→A`	AC, BC	**ja**	**ja**	alle Attribute prim
`ABCD, AB→C, C→D, D→AB`	AB, C, D	**ja**	**ja**	alle prim

---

5 · Synthesealgorithmus (→ 3NF)

Liefert eine Zerlegung, die **verbundtreu** (verlustfrei), **abhängigkeitstreu** und in **3NF** ist.

Schritt 0: Kanonische Überdeckung F_c

```
1. Linksreduktion:  X → Y, A ∈ X:  Y ⊆ (X–A)⁺ ? ⇒ A streichen
2. Rechtsreduktion: X → Y, B ∈ Y:  B ∈ X⁺ ohne diese FD ? ⇒ B streichen
3. X → ∅ entfernen
4. Vereinigung: X → Y, X → Z  ⇒  X → YZ
```

Algorithmus

1. F_c bestimmen.
2. Für jede FD `X → Y` eine Relation $R_i(\underline{X}, Y)$.
3. Enthält keine Relation einen Schlüssel von R → Relation mit dem **Schlüssel** dazu.
4. Relationen streichen, die in einer anderen ganz enthalten sind.

Beispiel 1 Test 2025

R = ABCDE, F = {A → CD, BC → E} (schon kanonisch)

```
Schlüssel: AB  (B nie rechts, A nie rechts; AB⁺ = ABCDE)
R1(A, C, D)    aus A → CD
R2(B, C, E)    aus BC → E
R3(A, B)       Schlüssel fehlt noch
```

Beispiel 2 Skript 3.8

R = ABCD, F = {A→D, B→A, B→C, C→D, A→B}

```
Rechtsreduktion: A→D überflüssig (A→B→C→D)
F_c = {A→B, B→AC, C→D}
R1(A,B) ⊆ R2(A,B,C) ⇒ streichen
Ergebnis: R2(B, A, C)  [auch A ist Schlüssel],  R3(C, D)
Schlüssel von R (A bzw. B) steckt in R2 ✓
```

Beispiel 3 Skript 3.8

R = ABCDE, F = {AE → ABCDE, A → BC, E → D}

```
AE→ABCDE: A,E trivial; B,C über A→BC; D über E→D ⇒ AE→∅ ⇒ weg
F_c = {A→BC, E→D}
R1(A, B, C),  R2(E, D),  R3(A, E)
```

Beispiel 4 – Linksreduktion

R = ABC, F = {AB → C, B → C}

```
AB→C: B⁺ = BC ∋ C ⇒ A streichen ⇒ B→C (doppelt)
F_c = {B→C};  Schlüssel AB
R1(B, C),  R2(A, B)
```

6 · SQL-DDL (MySQL)

6.1 Datentypen

Typ	Wofür
`INT` / `INTEGER`, `SMALLINT`, `TINYINT`	Ganzzahl (TINYINT(1) als Boolean)
`DECIMAL(p,s)`	exakt, z. B. Geld `DECIMAL(10,2)` – **nicht** FLOAT
`FLOAT`, `DOUBLE`	Gleitkomma (gerundet)
`CHAR(n)` / `VARCHAR(n)`	feste / variable Textlänge
`TEXT`	langer Text
`DATE`, `TIME`, `DATETIME`, `TIMESTAMP`	Datum / Zeit
`ENUM('a','b')`	fixe Werteliste (MySQL)

6.2 CREATE TABLE

```
CREATE TABLE Kunde (
  KundenID   INT          AUTO_INCREMENT,
  SVNR       CHAR(10)     NOT NULL UNIQUE,
  Name       VARCHAR(50)  NOT NULL,
  Status     VARCHAR(4)   DEFAULT 'neu',
  Kontostand DECIMAL(10,2) CHECK (Kontostand >= 0),
  PRIMARY KEY (KundenID)
);

CREATE TABLE Bestellung (
  KundenID INT NOT NULL,
  Nummer   INT NOT NULL,
  Datum    DATE DEFAULT (CURRENT_DATE),
  PRIMARY KEY (KundenID, Nummer),                -- zusammengesetzt
  CONSTRAINT fk_best_kunde FOREIGN KEY (KundenID)
      REFERENCES Kunde(KundenID)
      ON DELETE CASCADE ON UPDATE CASCADE       -- Löschweitergabe
);
```

Constraint	Wirkung
`PRIMARY KEY`	eindeutig + NOT NULL, einmal pro Tabelle
`UNIQUE`	eindeutig (Schlüsselkandidat, z. B. SVNR)
`NOT NULL`	Pflichtfeld
`CHECK (bed)`	Wertebereich, z. B. `CHECK (zustand IN ('del','inst','plan'))`
`DEFAULT x`	Standardwert
`FOREIGN KEY … REFERENCES`	Verweis auf PK/UNIQUE der anderen Tabelle (Typen müssen gleich sein)
`ON DELETE CASCADE / SET NULL / RESTRICT`	was beim Löschen des Elternsatzes passiert

> Rekursive n:m-Beziehung (z. B. *benoetigt*): **zwei** FKs auf **dieselbe** Tabelle, mit verschiedenen Spaltennamen.

6.3 ALTER / DROP

```
ALTER TABLE software ADD speicherbedarf INT;                -- Spalte dazu
ALTER TABLE Ernte ADD Preis DECIMAL(8,2) CHECK (Preis > 0);
ALTER TABLE Ernte ADD CONSTRAINT chk_menge CHECK (Menge > 0);
ALTER TABLE Winzer DROP COLUMN StartJahr;                     -- Spalte weg
ALTER TABLE Kunde MODIFY Name VARCHAR(100) NOT NULL;          -- Typ ändern
ALTER TABLE Kunde RENAME COLUMN Name TO Nachname;
ALTER TABLE bevorzugt DROP FOREIGN KEY fk_bev_winzer;         -- FK weg
ALTER TABLE Bestellung DROP PRIMARY KEY,
                       ADD PRIMARY KEY (Datum, KundenID, Nummer);
DROP TABLE Geschenk;                                          -- Tabelle weg
```

6.4 Autoincrement in Oracle

```
-- bis 11g: Sequence
CREATE SEQUENCE geschenk_seq START WITH 1 INCREMENT BY 1;
INSERT INTO Geschenk (ID, KundenID, Beschreibung)
VALUES (geschenk_seq.NEXTVAL, 4711, 'Wein');

-- ab 12c: Identity
CREATE TABLE Geschenk (
  ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  KundenID NUMBER NOT NULL,
  Datum DATE DEFAULT SYSDATE
);
```

6.5 MySQL Workbench – Beziehungstypen

Typ	Bedeutung	Wann
Identifying (durchgezogen)	FK ist **Teil des PK** des Kindes	Weak Entity (1:n), **IS-A / Subtyp (1:1)**, n:m-Tabelle
Non-Identifying (strichliert)	FK ist normales Attribut	normale 1:1 / 1:n

> IST-Beziehung A ← B ⇒ B bekommt `a1` als PK **und** FK ⇒ **1:1 Identifying Relationship**. (Test 2024 A3)

---

7 · Übungen 1–4 mit Lösungen

Übung 1 – EER-Modellierung

Angaben A (Firma/Projekte), B (Lehrveranstaltung Datenmodellierung), C (Versicherung) stehen in `insy4_uebung1.pdf`. Erst selbst zeichnen!

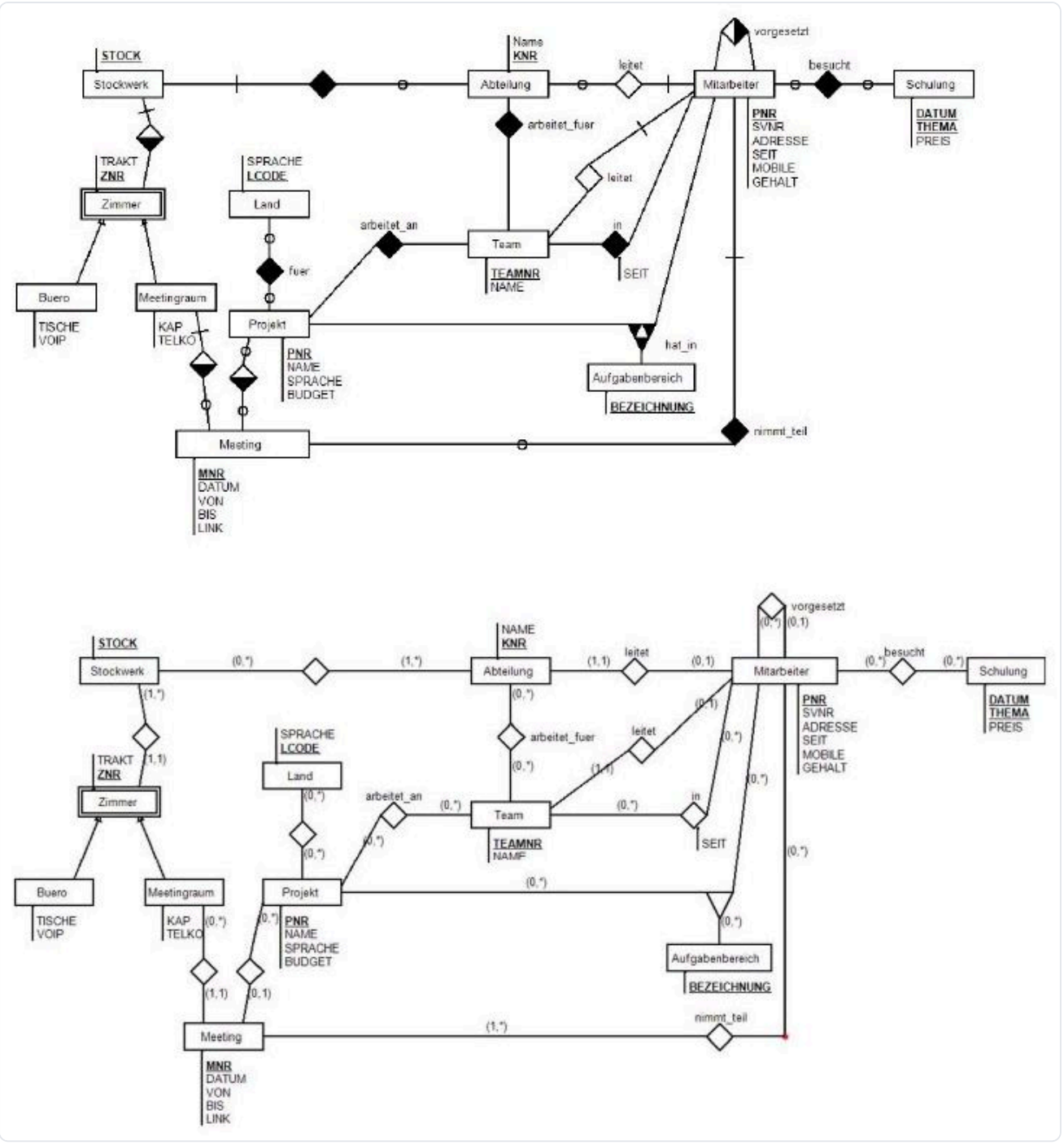
Wichtige Modellierungsentscheidungen

▼ A – Kernpunkte

- **Zimmer** = Weak Entity von Stockwerk (ZNR nur pro Stock eindeutig) → Schlüssel (STOCK, ZNR).
- Zimmer ← Büro / Meetingraum (Generalisierung, disjunkt).
- Mitarbeiter *leitet* Abteilung (1:1) und *leitet* Team (1:1); *vorgesetzt* rekursiv 1:n.

- Mitarbeiter *in* Team: n:m mit Attribut SEIT. *besucht* Schulung: n:m.
- *hat_in*: ternär Mitarbeiter – Projekt – Aufgabenbereich.
- Meeting: 1:n zu Meetingraum (zwingend), 1:n zu Projekt (optional!), *nimmt_teil* n:m.

▾ Lösung A (Teorey + min-max)



▾ B – Kernpunkte

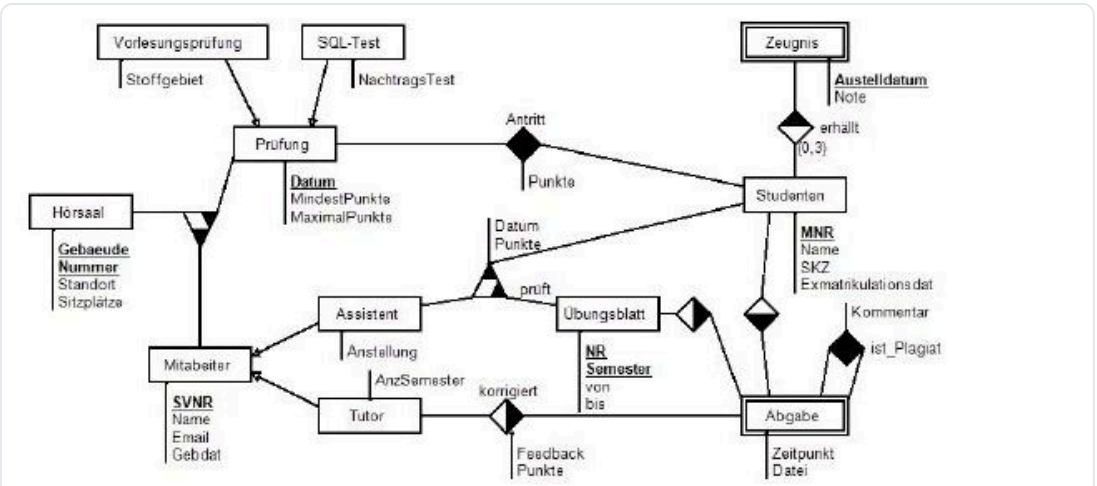
- Mitarbeiter ← Assistent / Tutor; Prüfung ← SQL-Test / Vorlesungsprüfung.
- **Abgabe** = Weak Entity (Übungsblatt + Student). *korrigiert*: Tutor 1:n Abgabe (genau ein Tutor), Attribute FEEDBACK, PUNKTE.
- *Plagiat*: rekursiv n:m auf Abgabe mit KOMMENTAR.

- *prüft*: ternär Assistent – Student – Übungsblatt (PUNKTE, DATUM).
- *Aufsicht*: ternär Mitarbeiter – Hörsaal – Prüfung. *tritt an*: Student n:m Prüfung (PUNKTE).
- **Zeugnis** = Weak Entity von Student, (0,3) bei Student.

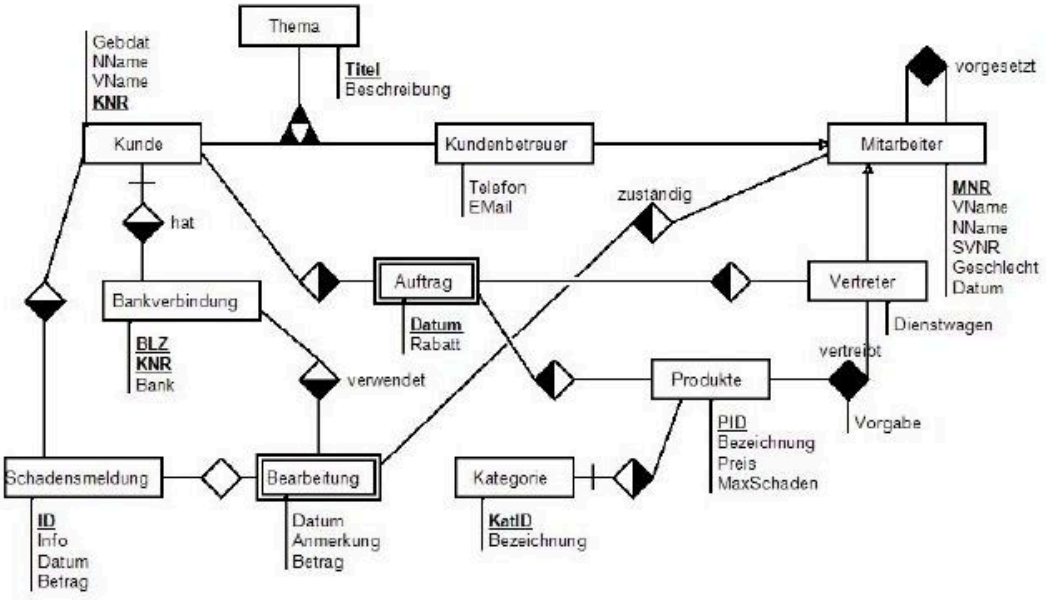
▾ C – Kernpunkte

- Mitarbeiter ← Vertreter / Kundenbetreuer; *Vorgesetzter* rekursiv **n:m**.
- Produkt n:1 Kategorie; Vertreter n:m Produkt mit VORGABE.
- **Auftrag** = Weak Entity (Kunde + Produkt + DATUM), Attribut RABATT, Beziehung zu Vertreter.
- **Bankverbindung** (BLZ, KNR) gehört zu genau einem Kunden (1:n).
- *berät*: ternär Kundenbetreuer – Kunde – Thema.
- Schadensmeldung 1:n Kunde; **Bearbeitung** = Weak Entity von Schadensmeldung, Beziehungen zu Bankverbindung und Mitarbeiter.

▾ Lösung B und C (Diagramme)

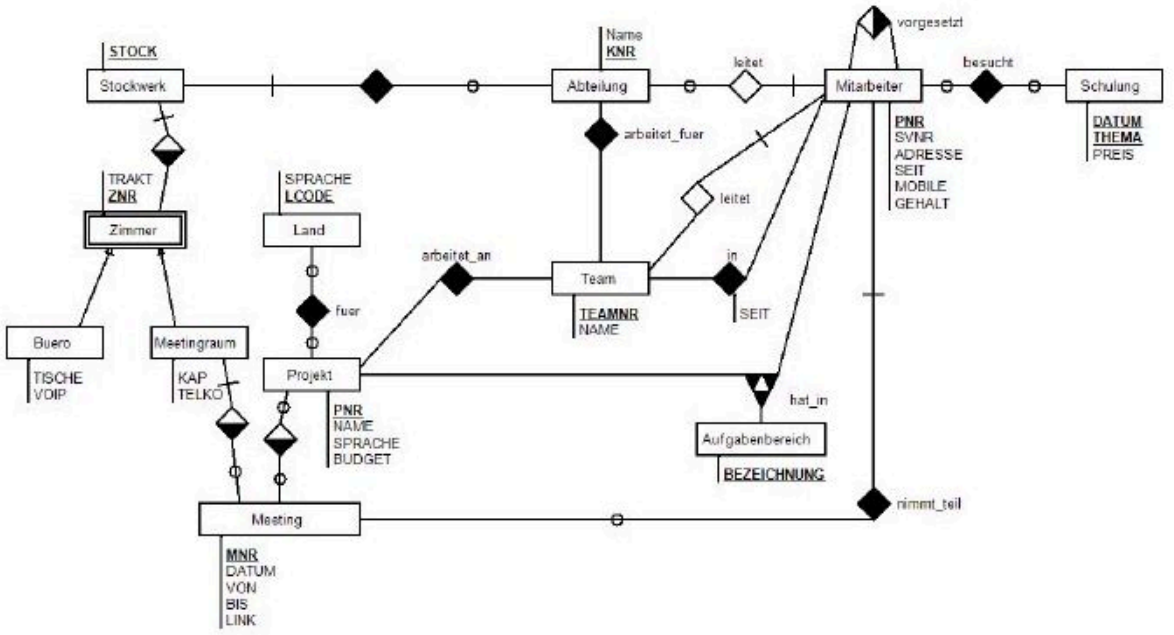


LÖSUNG C



Übung 2 – EER → Relationenmodell + 3NF

A – Firma (aus Übung 1A)



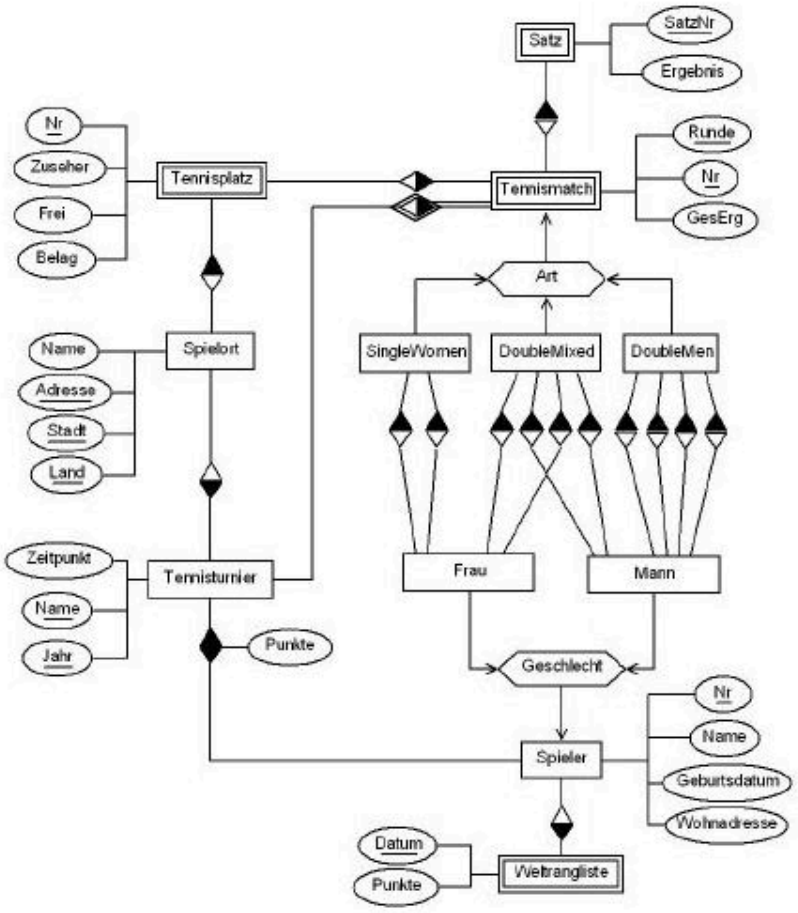
▾ Lösung A

```
Stockwerk         (STOCK)
Abteilung         (KNR, NAME, #LeiterPNR)
Abteilung_Stock   (#STOCK, #KNR)
Mitarbeiter       (PNR, SVNR, ADRESSE, SEIT, MOBILE, GEHALT, #VorgesetztPNR)
Schulung          (DATUM, THEMA, PREIS)
besucht           (#PNR, #DATUM, #THEMA)
Team              (TEAMNR, NAME, #LeiterPNR)
in                (#PNR, #TEAMNR, SEIT)
Zimmer            (#STOCK, ZNR, TRAKT)
Buero             (#STOCK, #ZNR, TISCHE, VOIP)
Meetingraum       (#STOCK, #ZNR, KAP, TELKO)
Land              (LCODE, SPRACHE)
Projekt           (PNR, NAME, SPRACHE, BUDGET)
fuer              (#LCODE, #PNR)
Meeting           (MNR, DATUM, VON, BIS, LINK, #ZNR, #STOCK, #ProjektPNR)
arbeitet_fuer     (#TEAMNR, #KNR)
arbeitet_an       (#TEAMNR, #PNR)
Aufgabenbereich   (BEZEICHNUNG)
hat_in            (#ProjektPNR, #BEZEICHNUNG, #MitarbeiterPNR)
nimmt_teil        (#MNR, #PNR)
```

3NF: alle Relationen erfüllen 3NF (jede FD hat einen Schlüssel links). SVNR ist zusätzlicher Schlüssel von Mitarbeiter (UNIQUE).

B – Tennisweltverband

▾ Lösung B



Spielort	(Name, Adresse, Stadt, Land	)
Tennisplatz	(Adresse, Stadt, Land, Nr, Zuseher, Frei, Belag	)
Tennisturnier	(Name, Jahr, Zeitpunkt, Adresse, Stadt, Land	)
Tennismatch	(Name, Jahr, Runde, Nr, Art, Adresse, Stadt, Land, PlatzNr, GesErg	)
Satz	(Name, Jahr, Runde, Nr, SatzNr, Ergebnis	)
Spieler	(Nr, Name, Geburtsdatum, Wohnadresse, Geschlecht	)
Mann	(Nr	)
Frau	(Nr	)
SingleWomen	(Name, Jahr, Runde, Nr, F1, F2	)
DoubleMixed	(Name, Jahr, Runde, Nr, T1M, T1F, T2M, T2F	)
DoubleMen	(Name, Jahr, Runde, Nr, T1M1, T1M2, T2M1, T2M2	)
Tennisturnier_Spieler	(Name, Jahr, Nr, Punkte	)
Weltrangliste	(Nr, Datum, Punkte	)

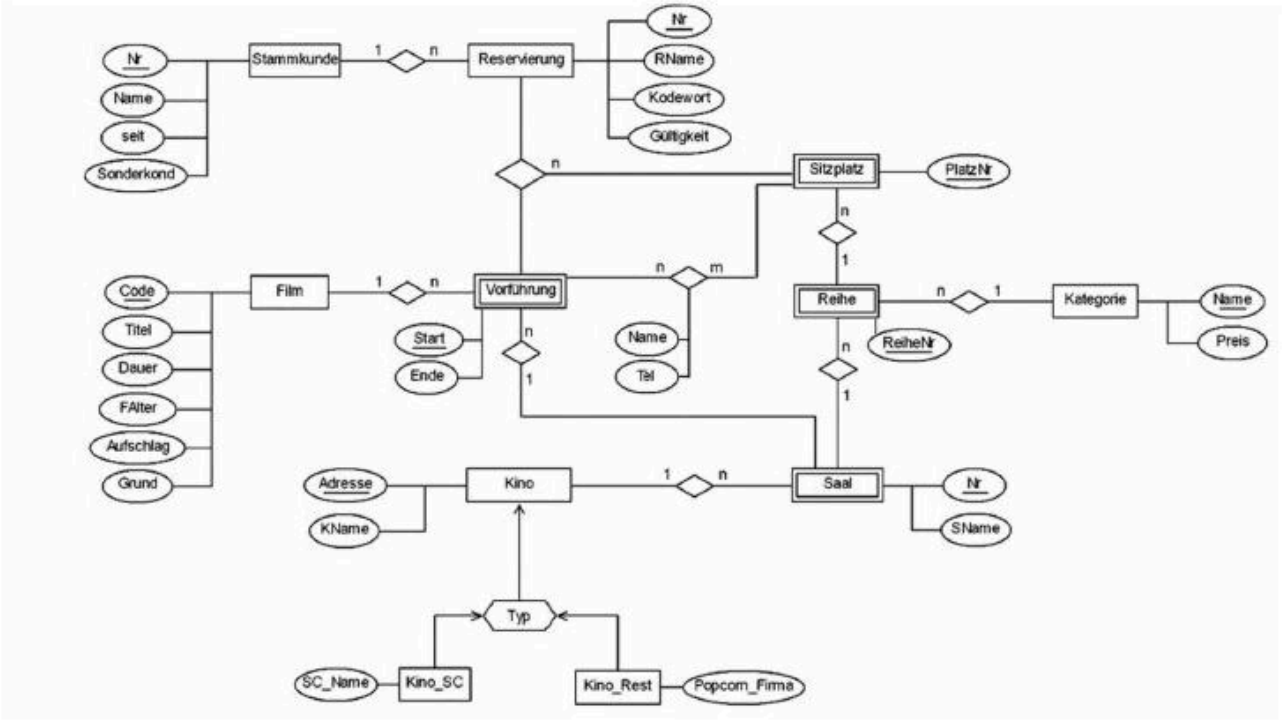
Anmerkung: Frau1 (F1), Team1Mann (T1M), Team2Frau (T2F),... sind ForeignKeys auf Nr bei Mann bzw. Frau

```
Spielort        (Adresse, Stadt, Land, Name)
Tennisplatz     (#Adresse, #Stadt, #Land, Nr, Zuseher, Frei, Belag)
Tennisturnier   (Name, Jahr, Zeitpunkt, #Adresse, #Stadt, #Land)
Tennismatch     (#Name, #Jahr, Art, Runde, Nr, #Adresse, #Stadt, #Land, #PlatzN
Satz            (#Name, #Jahr, #Art, #Runde, #Nr, SatzNr, Ergebnis)
Spieler         (Nr, Name, Geburtsdatum, Wohnadresse, Geschlecht)
Mann            (#Nr)       Frau (#Nr)
SingleWomen     (#Name, #Jahr, #Runde, #Nr, #F1, #F2)
DoubleMixed     (#Name, #Jahr, #Runde, #Nr, #T1M, #T1F, #T2M, #T2F)
DoubleMen       (#Name, #Jahr, #Runde, #Nr, #T1M1, #T1M2, #T2M1, #T2M2)
Turnier_Spieler (#Name, #Jahr, #SpielerNr, Punkte)
Weltrangliste   (#SpielerNr, Datum, Punkte)
```

F1, T1M … = FKs auf Frau.Nr bzw. Mann.Nr. 3NF-Falle: Wohnadresse mit PLZ → Ort wäre transitiv (eure Gruppe hat das mit `Ort(PLZ, Ort)` sauber gelöst).

C – Kinokette

▼ Lösung C



Kino	(Adresse, KName, Typ	)
KinoSC	(Adresse, SCName	)
KinoRest	(Adresse, PopcornFirma	)
Saal	(Adresse, Nr, SName	)
Reihe	(Adresse, Nr, ReiheNr, KategorieName	)
Sitzplatz	(Adresse, Nr, ReiheNr, PlatzNr	)
Kategorie	(Name, Preis	)
Stammkunde	(Nr, Name, seit, Sonderkond	)
Film	(Code, Titel, Dauer, FAlter, Aufschlag, Grund	)
Vorfuehrung	(Code, Adresse, Nr, Start, Ende	)
Reservierung	(Nr, RName, Kodewort, Gueltigkeit, StammkundenNr	)
Res_Vorf_Sitz	(ReservierungsNr,Adresse, Nr, ReiheNr, PlatzNr, Start	)
Vorf_Sitz	(Adresse, Nr, Start, ReiheNr, PlatzNr, Name, Tel	)

```
Kino          (Adresse, KName, Typ)
KinoSC        (#Adresse, SCName)
KinoRest      (#Adresse, PopcornFirma)
Saal          (#Adresse, Nr, SName)
Reihe         (#Adresse, #Nr, ReiheNr, #KategorieName)
Sitzplatz     (#Adresse, #Nr, #ReiheNr, PlatzNr)
Kategorie     (Name, Preis)
Stammkunde    (Nr, Name, seit, Sonderkond)
Film          (Code, Titel, Dauer, FAlter, Aufschlag, Grund)
Vorfuehrung   (#Adresse, #Nr, Start, #Code, Ende)
Reservierung  (Nr, RName, Kodewort, Gueltigkeit, #StammkundenNr)
Res_Vorf_Sitz (#Adresse, #Nr, #ReiheNr, #PlatzNr, #Start, #ReservierungsNr)
Vorf_Sitz     (#Adresse, #Nr, #Start, #ReiheNr, #PlatzNr, Name, Tel)
```

Doppelverkauf wird verhindert, weil (Saal, Start, Sitz) Schlüssel ist. Kategorie als eigene Relation, sonst Preis transitiv über Kategorie → keine 3NF.

Übung 3 – Normalformen

A – Schlüssel + NF

R	F
ABC	A→BC
ABC	A→B
ABC	A→B, B→A
ABCDEF	EF→E, ABC→E, AB→C
ABCDEF	A→BC, D→EF
ABCD	A→BC, BC→D, D→A

▼ Lösung A

F	Schlüssel	2NF	3NF	Begründung
A→BC	A	ja	ja	Schlüssel links
A→B	AC	nein	nein	A ⊊ AC → B
A→B, B→A	AC, BC	ja	ja	alle prim
EF→E, ABC→E, AB→C	ABDF	nein	nein	AB ⊊ ABDF → C (EF→E trivial)
A→BC, D→EF	AD	nein	nein	A → BC partiell
A→BC, BC→D, D→A	A, D, BC	ja	ja	alle prim

B – 3NF-Zerlegung

R	F
ABCD	A→B, B→CD
ABCD	A→B
ABC	AB→C, B→C
ABCDEF	E→F, AB→CDE
ABCDEF	E→F, B→CDE
ABCDE	AB→CDE

▼ Lösung B

F	Schlüssel	Zerlegung
A→B, B→CD	A	R1(A,B) R2(B,C,D)
A→B	ACD	R1(A,B) R2(A,C,D)
AB→C, B→C	AB	F_c={B→C}: R1(B,C) R2(A,B)
E→F, AB→CDE	AB	R1(E,F) R2(A,B,C,D,E)
E→F, B→CDE	AB	R1(E,F) R2(B,C,D,E) R3(A,B)
AB→CDE	AB	keine Zerlegung nötig – schon 3NF

C – Wein(Bezeichnung, Lieferant, LieferantOrt, Erzeuger, Traubenart, Geschmack, Preis)

▼ Lösung C

```
1. Lieferant → LieferantOrt
   Bezeichnung, Lieferant → Preis, Erzeuger
   Bezeichnung → Traubenart
   Traubenart → Geschmack
   Schlüssel: (Bezeichnung, Lieferant)
```

2. Nur **1NF**: LieferantOrt hängt nur von Lieferant ab, Traubenart nur von Bezeichnung (Teil des Schlüssels) → 2NF verletzt; Geschmack transitiv über Traubenart → 3NF verletzt.
3. 3NF:

```
Lieferant (Lieferant, LieferantOrt)
Wein      (Bezeichnung, #Traubenart)
Traube    (Traubenart, Geschmack)
Angebot   (#Bezeichnung, #Lieferant, Erzeuger, Preis)
```

D – Inventar(InvNr, Beschreibung, AJahr, RaumNr, RaumBez)

▼ Lösung D

```
InvNr → Beschreibung, AJahr, RaumNr, RaumBez
RaumNr → RaumBez
```

2. **2NF ja** – Schlüssel InvNr ist einelementig, keine partielle Abhängigkeit möglich.
3. **3NF nein** – InvNr → RaumNr → RaumBez (transitiv).
4. Problem: Raum 1304 umbenennen → 3 Zeilen ändern (Update-Anomalie); letztes Gerät aus 1310 löschen → „Büro einzel“ für 1310 weg (Delete-Anomalie); neuer Raum ohne Gerät nicht speicherbar (Insert-Anomalie).
5. `Inventar(InvNr, Beschreibung, AJahr, #RaumNr)` + `Raum(RaumNr, RaumBez)`

E – Telefonliste(SVNR, Name, RaumNr, Raumbezeichnung, Telefonklappe)

▼ Lösung E

```
SVNR → Name
RaumNr → Raumbezeichnung
Telefonklappe → SVNR, RaumNr   (⇒ Name, Raumbezeichnung)
```

2. Schlüssel: **Telefonklappe**. (SVNR nicht – M. Greis hat zwei Klappen in Raum 15 und 16.)
3. Anomalie: M. Greis heiratet → Name in 2 Zeilen ändern; Raum 16 von „Buero“ auf „Labor“ → 2 Zeilen (Update-Anomalie).
4. **2NF ja** (einelementiger Schlüssel), aber **3NF nein** (Name und Raumbezeichnung transitiv).

Übung 4.1 – SQL-DDL

B – CREATE TABLE für die Wein-Datenbank

Annahme Schlüssel (Unterstreichung ging im PDF-Text verloren): Ernte = (WeinsorteName, WinzerCode, ErnteJahr), Bestellung = (KundenID, Nummer), Geschenk = (KundenID, Datum).

▼ Lösung B

```
CREATE TABLE Weinsorte (
  WeinsorteName VARCHAR(50) PRIMARY KEY,
  Anmerkung     VARCHAR(255)
);
CREATE TABLE Winzer (
  WinzerCode INT PRIMARY KEY,
  Name       VARCHAR(50) NOT NULL,
  Adresse    VARCHAR(100),
  StartJahr  INT
);
CREATE TABLE Ernte (
  WeinsorteName VARCHAR(50),
  WinzerCode    INT,
  ErnteJahr     INT,
  Menge         INT,
  PRIMARY KEY (WeinsorteName, WinzerCode, ErnteJahr),
  FOREIGN KEY (WeinsorteName) REFERENCES Weinsorte(WeinsorteName),
  FOREIGN KEY (WinzerCode)    REFERENCES Winzer(WinzerCode)
);
CREATE TABLE Kunde (
  KundenID   INT PRIMARY KEY,
  Name       VARCHAR(50) NOT NULL,
  Adresse    VARCHAR(100),
  Kontostand DECIMAL(10,2)
);
CREATE TABLE bevorzugt (
  KundenID      INT,
  WeinsorteName VARCHAR(50),
  WinzerCode    INT,
  PRIMARY KEY (KundenID, WeinsorteName, WinzerCode),
  FOREIGN KEY (KundenID)      REFERENCES Kunde(KundenID),
  FOREIGN KEY (WeinsorteName) REFERENCES Weinsorte(WeinsorteName),
  CONSTRAINT fk_bev_winzer FOREIGN KEY (WinzerCode) REFERENCES Winzer(WinzerCo
);
CREATE TABLE Bestellung (
  KundenID      INT,
  Nummer        INT,
  WeinsorteName VARCHAR(50) NOT NULL,
  WinzerCode    INT NOT NULL,
  ErnteJahr     INT NOT NULL,
  Anzahl        INT,
  PRIMARY KEY (KundenID, Nummer),
  FOREIGN KEY (KundenID) REFERENCES Kunde(KundenID),
  FOREIGN KEY (WeinsorteName, WinzerCode, ErnteJahr)
      REFERENCES Ernte(WeinsorteName, WinzerCode, ErnteJahr)
);
CREATE TABLE Geschenk (
  KundenID     INT,
  Datum        DATE,
  Beschreibung VARCHAR(255),
  PRIMARY KEY (KundenID, Datum),
```

```
  FOREIGN KEY (KundenID) REFERENCES Kunde(KundenID)
);
```

C – Tabellen ändern

▼ Lösung C

```
ALTER TABLE Ernte ADD CONSTRAINT chk_menge CHECK (Menge > 0);

ALTER TABLE Ernte ADD Preis DECIMAL(8,2) CHECK (Preis > 0);

ALTER TABLE Winzer DROP COLUMN StartJahr;

ALTER TABLE Bestellung ADD Datum DATE NOT NULL;
ALTER TABLE Bestellung DROP PRIMARY KEY,
                       ADD PRIMARY KEY (Datum, KundenID, Nummer);

ALTER TABLE bevorzugt DROP FOREIGN KEY fk_bev_winzer;
ALTER TABLE bevorzugt DROP PRIMARY KEY;
ALTER TABLE bevorzugt DROP COLUMN WinzerCode;
ALTER TABLE bevorzugt ADD PRIMARY KEY (KundenID, WeinsorteName);

DROP TABLE Geschenk;

CREATE TABLE Geschenk (
  ID           INT AUTO_INCREMENT PRIMARY KEY,
  KundenID     INT NOT NULL,
  Datum        DATE DEFAULT (CURRENT_DATE),
  Beschreibung VARCHAR(255),
  FOREIGN KEY (KundenID) REFERENCES Kunde(KundenID)
);
```

Problem bei bevorzugt: Bevorzugt ein Kunde dieselbe Sorte von zwei Winzern, entstehen nach dem Entfernen doppelte (KundenID, WeinsorteName) → neuer PK schlägt fehl bzw. Information geht verloren. Außerdem muss zuerst der FK auf WinzerCode entfernt werden.

Ältere MySQL-Versionen: statt `DEFAULT (CURRENT_DATE)` → `Datum TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.

D – Krankenhaus (mit Löschweitergabe)

▼ Lösung D

```
CREATE TABLE Krankenhaus (
  ID   INT PRIMARY KEY,
  NAME VARCHAR(100) NOT NULL
);
CREATE TABLE Station (
  KH_ID  INT,
  STNR   INT,
  BEZ    VARCHAR(50),
  BETTEN INT CHECK (BETTEN >= 0),
  PRIMARY KEY (KH_ID, STNR),
  FOREIGN KEY (KH_ID) REFERENCES Krankenhaus(ID) ON DELETE CASCADE
);
CREATE TABLE Mitarbeiter (
  ID       INT PRIMARY KEY,
  SVNR     CHAR(10) NOT NULL UNIQUE,
  VORNAME  VARCHAR(50),
  NACHNAME VARCHAR(50),
  GEHALT   DECIMAL(10,2)          -- exakt, 2 Kommastellen
);
CREATE TABLE arbeitet_in (
  MA_ID   INT,
  KH_ID   INT,
  STNR    INT,
  STUNDEN INT,
  PRIMARY KEY (MA_ID, KH_ID, STNR),
  FOREIGN KEY (MA_ID) REFERENCES Mitarbeiter(ID) ON DELETE CASCADE,
  FOREIGN KEY (KH_ID, STNR) REFERENCES Station(KH_ID, STNR) ON DELETE CASCADE
);
```

Station = Weak Entity (1:n identifying), arbeitet_in = n:m mit Attribut STUNDEN.

E – Autonummerierung in Oracle

▼ Lösung E

Siehe 6.4: `CREATE SEQUENCE … / seq.NEXTVAL` (bis 11g) oder `GENERATED ALWAYS AS IDENTITY` (ab 12c).

Skript-Beispiele (3.6 Normalformen)

Schlüssel + NF

#	R, F
1	`ABCDE, A→B`
2	`ABCDEF, A→B, F→A, C→D`
3	`ABCD, A→D, B→A, B→C, C→D, A→B`
4	`ABCDE, A→BC, E→D`
5	`ABCD, AB→C, C→D, D→AB`
6	`ABC, B→AC`

▼ Lösung

#	Schlüssel	2NF	3NF	Begründung
1	ACDE	**nein**	**nein**	A ⊊ Schlüssel → B
2	CEF	**nein**	**nein**	F→A, C→D partiell
3	A, B	**ja**	**nein**	einelementig; C→D transitiv
4	AE	**nein**	**nein**	A→BC partiell
5	AB, C, D	**ja**	**ja**	alle prim (sogar BCNF)
6	B	**ja**	**ja**	Schlüssel links

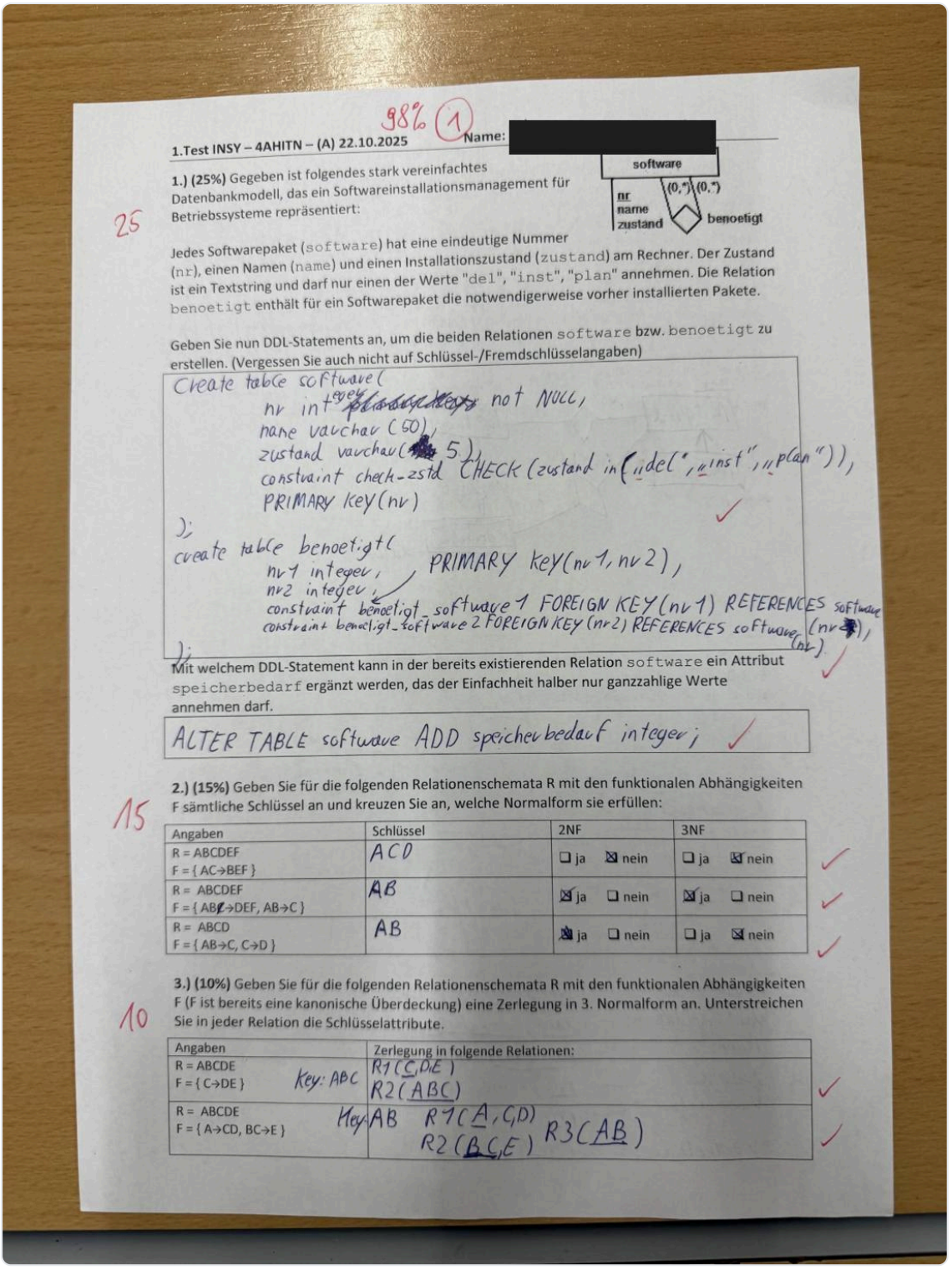
---

8 · Alte Tests – gelöst und erklärt

Die gleichen Lösungen gibt es auch als PDF (`Test_2024_Loesung.pdf` , `Test_2025_Loesung.pdf`).

Test 22.10.2025 (Seite 1 = Aufgaben 1–3)

▼ Original-Foto anzeigen



98% (1)

1.Test INSY – 4AHITN – (A) 22.10.2025 Name: [redacted]

25

1.) (25%) Gegeben ist folgendes stark vereinfachtes Datenbankmodell, das ein Softwareinstallationsmanagement für Betriebssysteme repräsentiert:

Jedes Softwarepaket (`software`) hat eine eindeutige Nummer (`nr`), einen Namen (`name`) und einen Installationszustand (`zustand`) am Rechner. Der Zustand ist ein Textstring und darf nur einen der Werte `"del"`, `"inst"`, `"plan"` annehmen. Die Relation `benoetigt` enthält für ein Softwarepaket die notwendigerweise vorher installierten Pakete.

Geben Sie nun DDL-Statements an, um die beiden Relationen `software` bzw. `benoetigt` zu erstellen. (Vergessen Sie auch nicht auf Schlüssel-/Fremdschlüsselangaben)

```
Create table software(
    nr integer not NULL,
    name varchar(50),
    zustand varchar(5),
    constraint check_zstd CHECK (zustand in ("del","inst","plan")),
    PRIMARY key(nr)
);
create table benoetigt(
    nr1 integer,
    nr2 integer,
    PRIMARY key(nr1, nr2),
    constraint benoetigt_software1 FOREIGN KEY(nr1) REFERENCES software(nr),
    constraint benoetigt_software2 FOREIGN KEY(nr2) REFERENCES software(nr)
);
```

Mit welchem DDL-Statement kann in der bereits existierenden Relation `software` ein Attribut `speicherbedarf` ergänzt werden, das der Einfachheit halber nur ganzzahlige Werte annehmen darf.

```
ALTER TABLE software ADD speicherbedarf integer;
```

15

2.) (15%) Geben Sie für die folgenden Relationenschemata R mit den funktionalen Abhängigkeiten F sämtliche Schlüssel an und kreuzen Sie an, welche Normalform sie erfüllen:

Angaben	Schlüssel	2NF	3NF
R = ABCDEF F = { AC→BEF }	ACD	☐ ja ☒ nein	☐ ja ☒ nein
R = ABCDEF F = { AB→DEF, AB→C }	AB	☒ ja ☐ nein	☒ ja ☐ nein
R = ABCD F = { AB→C, C→D }	AB	☒ ja ☐ nein	☐ ja ☒ nein

10

3.) (10%) Geben Sie für die folgenden Relationenschemata R mit den funktionalen Abhängigkeiten F (F ist bereits eine kanonische Überdeckung) eine Zerlegung in 3. Normalform an. Unterstreichen Sie in jeder Relation die Schlüsselattribute.

Angaben	Zerlegung in folgende Relationen:
R = ABCDE F = { C→DE } Key: ABC	R1(C,DE) R2(ABC)
R = ABCDE F = { A→CD, BC→E } Key: AB	R1(A,CD) R2(BC,E) R3(AB)

1) 25 % – DDL für software / benoetigt SQL

software(nr, name, zustand ∈ {del, inst, plan}), benoetigt = rekursive n:m-Beziehung (0,*)/(0,*) auf software.

▼ Lösung

```
CREATE TABLE software (
  nr      INT NOT NULL,
  name    VARCHAR(50),
  zustand VARCHAR(4),
  CONSTRAINT chk_zustand CHECK (zustand IN ('del','inst','plan')),
  PRIMARY KEY (nr)
);

CREATE TABLE benoetigt (
  nr           INT NOT NULL,   -- Paket
  benoetigt_nr INT NOT NULL,   -- vorher nötiges Paket
  PRIMARY KEY (nr, benoetigt_nr),
  FOREIGN KEY (nr)           REFERENCES software(nr),
  FOREIGN KEY (benoetigt_nr) REFERENCES software(nr)
);

ALTER TABLE software ADD speicherbedarf INT;
```

Was man können muss: n:m → eigene Tabelle; rekursiv → 2 FKs auf dieselbe Tabelle; Wertebereich → CHECK … IN; Spalte nachträglich → ALTER TABLE … ADD.

2) 15 % – Schlüssel + 2NF/3NF

R	F
ABCDEF	AC→BEF
ABCDEF	AB→DEF, AB→C
ABCD	AB→C, C→D

▼ Lösung

F	Schlüssel	2NF	3NF	Rechnung
AC→BEF	ACD	nein	nein	D nie rechts ⇒ ACD^+=ABCDEF; AC ⊊ ACD → BEF
AB→DEF, AB→C	AB	ja	ja	AB→CDEF, nur Schlüssel links
AB→C, C→D	AB	ja	nein	AB→C→D transitiv

3) 10 % – 3NF-Zerlegung

R	F
ABCDE	C→DE
ABCDE	A→CD, BC→E

▼ Lösung

```
C→DE:        Schlüssel ABC  ⇒  R1(C, D, E)   R2(A, B, C)
A→CD, BC→E:  Schlüssel AB   ⇒  R1(A, C, D)   R2(B, C, E)   R3(A, B)
```

Seite 2 (ab Aufgabe 4, 50 %) liegt nicht vor – laut Aufbau 2024 vermutlich: FDs aus Tabelle, EER → Relationen, Notation umrechnen. Alles in Kapitel 1–3 abgedeckt.

Test 24.10.2024 (alle Aufgaben)

▼ Original-Fotos anzeigen

96% (1)

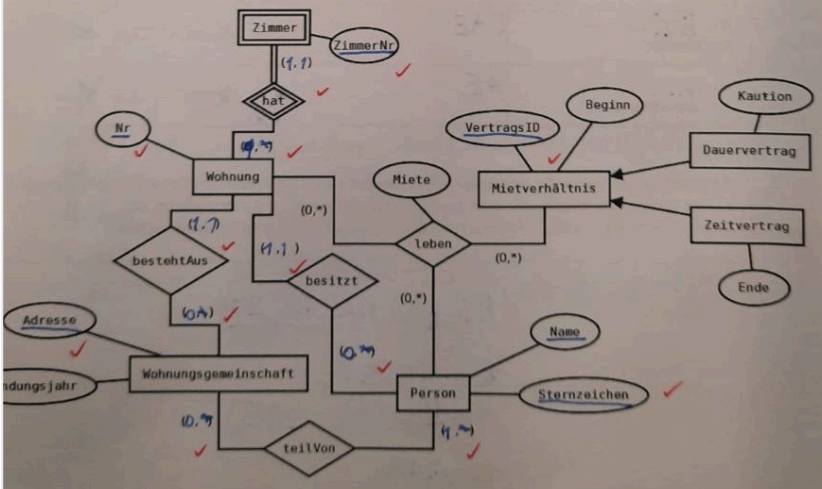
1.Test INSY – 4AHITN – (A) 24.10.2024 Name: [redacted]

1.) (26%) Ergänzen Sie für den nachfolgenden Sachverhalt das bereits begonnene EER-Diagramm. Beachten Sie, dass bei bereits eingezeichneten Beziehungen, teilweise noch die **min-max-Angaben** fehlen. Kennzeichnen Sie weiters auch fehlende Schlüsselangaben durch **Unterstreichen der Schlüsselattribute**.

Personen sind eindeutig gekennzeichnet durch die Kombination aus ihrem Namen und ihrem Sternzeichen. Manche Personen besitzen Wohnungen, einige besitzen sogar mehrere Wohnungen.
Eine Wohnung ist eindeutig gekennzeichnet durch eine Nummer. Jede Wohnung besteht aus einer Reihe an Zimmern. Zimmer sind eindeutig gekennzeichnet durch die Kombination von der Nummer der Wohnung und ihrer eigenen Zimmer-Nummer. Jede Wohnung besteht aus zumindest einem Zimmer. In dieser Datenbank ist auch jede Wohnung eindeutig zu einer Person zugeordnet, die diese Wohnung besitzt.

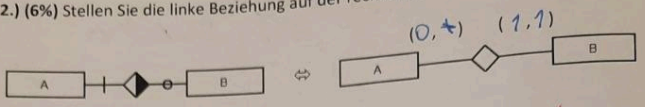
Mietverhältnisse sind eindeutig gekennzeichnet durch ihre VertragsID. Daneben wird auch der Beginn des Mietverhältnisses als Attribut festgehalten. Es wird unterschieden zwischen Dauerverträgen und Zeitverträgen, welches beides Formen von Mietverhältnissen sind. Bei Dauerverträgen wird auch die Kaution als Attribut festgehalten.
Bei Zeitverträgen gibt es zusätzlich ein Attribut, welches das formale Ende des Mietverhältnisses ausdrückt.
Die Relation „leben" drückt aus, welche Personen in welchen Wohnungen zu welchen Mietverhältnissen leben. Dazu wird auch das Attribut „Miete" als Teil dieser Relation gespeichert.

Wohnungsgemeinschaften bestehen aus keiner oder auch mehreren Wohnungen, wobei jede Wohnung zu genau einer einzigen Wohnungsgemeinschaft zugeordnet wird. Jede Wohnungsgemeinschaft hat eine eindeutige Adresse. Das Gründungsjahr wird auch festgehalten. Es können daneben auch Personen zu einer Wohnungsgemeinschaft gehören. Jede Person muss in der Datenbank zu zumindest einer Wohnungsgemeinschaft gehören, kann aber Teil von beliebig vielen sein.



6

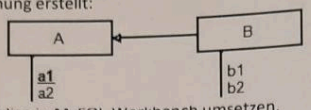
2.) (6%) Stellen Sie die linke Beziehung auf der rechten Seite mit Hilfe der (min,max)-Notation dar:



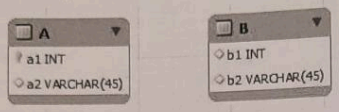
✓

4

3.) (4%) Jemand hat ein EER-Modell mit einer IST-Beziehung erstellt:



Sie sollen dies in MySQL-Workbench umsetzen.



Welchen Beziehungstyp zeichnen Sie zwischen A und B ein?

- ❑ 1:1 Non-Identifying Relationship
- ❑ 1:n Non-Identifying Relationship
- ❑ n:n Identifying Relationship
- ☒ 1:1 Identifying Relationship ✓
- ❑ 1:n Identifying Relationship

4

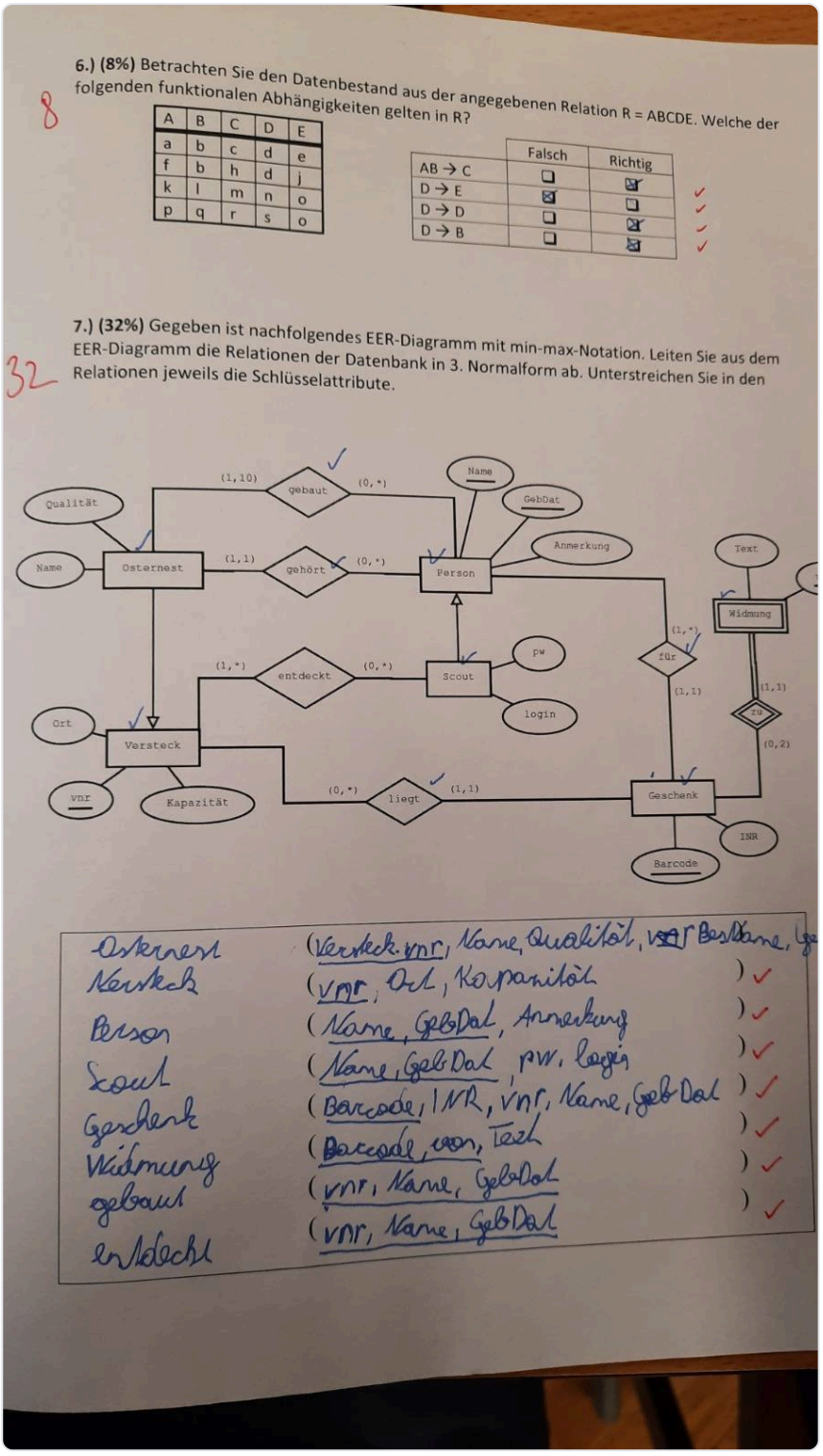
4.) (8%) Geben Sie für die folgenden Relationenschemata R mit den funktionalen Abhängigkeiten F sämtliche Schlüssel an und kreuzen Sie an, welche Normalform sie erfüllen:

Angaben	Schlüssel	2NF	3NF
R = ABCDEF F = {AE→BCF}	DAE	❑ ja ☒ nein	❑ ja ☒ nein
R = ABCDEF F = { ACE→BFD, AE→C }	AE	☒ ja ❑ nein X 20	❑ ja ☒ nein

16

5.) (16%) Geben Sie für die folgenden Relationenschemata R mit den funktionalen Abhängigkeite F (F ist bereits eine kanonische Überdeckung) eine Zerlegung in 3. Normalform an: Kennzeichnen Sie in allen Relationen den jeweiligen Schlüssel durch Unterstreichen.

Angaben	Zerlegung in folgende Relationen:
R = ABCDEF F = {B→D}	R1(BD) R2(ABCEF) Key: ABCEF ✓
R = ABCDE F = { AB→C, C→DE }	R_1(ABC) R_2(CDE) Key: AB ✓



6.) **(8%)** Betrachten Sie den Datenbestand aus der angegebenen Relation R = ABCDE. Welche der folgenden funktionalen Abhängigkeiten gelten in R?

8

A	B	C	D	E
a	b	c	d	e
f	b	h	d	j
k	l	m	n	o
p	q	r	s	o

	Falsch	Richtig
AB → C	☐	☑
D → E	☑	☐
D → D	☐	☑
D → B	☐	☑

7.) **(32%)** Gegeben ist nachfolgendes EER-Diagramm mit min-max-Notation. Leiten Sie aus dem EER-Diagramm die Relationen der Datenbank in 3. Normalform ab. Unterstreichen Sie in den Relationen jeweils die Schlüsselattribute.

32

Osternest (Versteck.vnr, Name, Qualität, ...)
Versteck (vnr, Ort, Kapazität)
Person (Name, GebDat, Anmerkung)
Scout (Name, GebDat, pw, login)
Geschenk (Barcode, INR, vnr, Name, GebDat)
Widmung (Barcode, von, Text)
gebaut (vnr, Name, GebDat)
entdeckt (vnr, Name, GebDat)

1) 26 % – EER-Diagramm ergänzen (Wohnungen) EER

Fehlende min-max-Angaben und Schlüssel einzeichnen (Text siehe Foto).

▼ Lösung

Schlüssel: Person(Name, Sternzeichen) · Wohnung(Nr) · Zimmer: Weak Entity, Teilschlüssel ZimmerNr · Mietverhältnis(VertragsID) · Wohnungsgemeinschaft(Adresse)

Beziehung	Entity	(min,max)	Textstelle
hat	Zimmer	(1,1)	Zimmer gehört zu genau einer Wohnung
	Wohnung	(1,*)	„zumindest einem Zimmer“
besitzt	Wohnung	(1,1)	„jede Wohnung eindeutig einer Person zugeordnet“
	Person	(0,*)	„manche … einige sogar mehrere“
bestehtAus	Wohnung	(1,1)	„genau einer einzigen WG zugeordnet“
	WG	(0,*)	„keiner oder auch mehreren Wohnungen“
teilVon	Person	(1,*)	„zumindest einer … beliebig vielen“
	WG	(0,*)	„es können … Personen gehören“

Dauervertrag (Kaution) und Zeitvertrag (Ende) sind Subtypen von Mietverhältnis. *leben* ist ternär (Wohnung, Person, Mietverhältnis) mit Attribut Miete.

2) 6 % – Teorey → (min,max)

`A –|– ◆ –o– B` (Hälfte zu B gefüllt)

▼ Lösung

```
| bei A ⇒ min(B)=1    o bei B ⇒ min(A)=0
gefüllte Hälfte zu B ⇒ max(A)=*,  leere Hälfte zu A ⇒ max(B)=1
⇒  A (0,*) –◇– (1,1) B
```

3) 4 % – IST-Beziehung in Workbench

B IS-A A, A(a1, a2), B(b1, b2). Welcher Beziehungstyp?

▼ Lösung

1:1 Identifying Relationship – B bekommt a1 als PK und FK (jedes B ist genau ein A).

4) 8 % – Schlüssel + 2NF/3NF

R	F
ABCDEF	AE→BCF
ABCDEF	ACE→BFD, AE→C

▼ Lösung

F	Schlüssel	2NF	3NF	Rechnung
AE→BCF	ADE	**nein**	**nein**	A,D,E nie rechts; AE ⊊ ADE → BCF
ACE→BFD, AE→C	AE	**ja**	**ja**	AE^+=AEC→ABCDEF; ACE ⊇ AE ist Superschlüssel

> Typischer Fehler (hier wurden 4 Punkte verloren): ACE→BFD sieht „transitiv“ aus, aber ACE enthält den Schlüssel AE ⇒ **Superschlüssel** ⇒ keine Verletzung ⇒ 3NF **ja**.

5) 16 % – 3NF-Zerlegung

R	F
ABCDEF	B→D
ABCDE	AB→C, C→DE

▼ Lösung

```
B→D:          Schlüssel ABCEF  ⇒  R1(B, D)   R2(A, B, C, E, F)
AB→C, C→DE:   Schlüssel AB     ⇒  R1(A, B, C)  R2(C, D, E)
```

6) 8 % – FDs aus Datenbestand

Tabelle und Lösung: siehe 3.2.

▼ Lösung

AB→C **richtig** · D→E **falsch** · D→D **richtig** · D→B **richtig**

7) 32 % – Osternest-EER → Relationen in 3NF

Diagramm siehe Foto Seite 3. Osternest IS-A Versteck; Scout IS-A Person; Widmung = Weak Entity von Geschenk.

▼ Lösung

```
Versteck  (vnr, Ort, Kapazität)
Osternest (#vnr, Name, Qualität, #BesitzerName, #BesitzerGebDat)   -- IS-A + ge
Person    (Name, GebDat, Anmerkung)
Scout     (#Name, #GebDat, pw, login)                              -- IS-A
Geschenk  (Barcode, INR, #vnr, #Name, #GebDat)                     -- liegt (1,1
Widmung   (#Barcode, von, Text)                                     -- Weak Ent
gebaut    (#vnr, #Name, #GebDat)                                   -- n:m (1,10)
entdeckt  (#vnr, #Name, #GebDat)                                   -- n:m Verste
```

Was man können muss: IS-A ⇒ Schlüssel vom Supertyp übernehmen; (1,1)-Seite bekommt den FK; n:m ⇒ eigene Relation; Weak Entity ⇒ Besitzer-Schlüssel + Teilschlüssel.

---

9 · Extra-Übungen (neu, im Test-Stil)

E1 – Schlüssel, NF, Zerlegung

R = ABCDE, F = {A→B, BC→D, D→E}

▼ Lösung

```
nie rechts: A, C  ⇒ AC⁺ = AC B D E = R  ⇒ Schlüssel AC
A→B: A ⊊ AC  ⇒ 2NF nein ⇒ 3NF nein
F_c = F  ⇒  R1(A,B)  R2(B,C,D)  R3(D,E)  R4(A,C)
```

E2 – zwei Schlüssel

R = ABCD, F = {AB→CD, D→A}

▼ Lösung

```
B nie rechts. AB⁺ = ABCD ✓, BD⁺ = BDAC ✓, BC⁺ = BC ✗
Schlüssel: AB, BD   nichtprim: C
AB→CD: Schlüssel ✓   D→A: A prim ✓
⇒ 2NF ja, 3NF ja  (BCNF nein: D ist kein Superschlüssel)
```

E3 – Kette

R = ABCDEF, F = {A→B, B→C, CD→EF}

▼ Lösung

```
nie rechts: A, D ⇒ AD⁺ = ADBCEF ⇒ Schlüssel AD
A→B partiell ⇒ 2NF nein, 3NF nein
R1(A,B)  R2(B,C)  R3(C,D,E,F)  R4(A,D)
```

E4 – Falle mit primen Attributen

R = ABCDE, F = {AB→C, C→A, D→E}

▼ Lösung

```
nie rechts: B, D.  BD⁺ = BDE ✗
ABD⁺ = ABDCE ✓   BCD⁺ = BCDAE ✓   ⇒ Schlüssel ABD, BCD
prim: A,B,C,D   nichtprim: E
D→E: D ⊊ Schlüssel, E nichtprim ⇒ 2NF nein, 3NF nein
```

E5 – FDs prüfen

A	B	C	D
1	x	p	5
2	x	q	5
3	y	p	6
4	y	q	6

A→B? B→D? C→B? BC→A? D→B?

▼ Lösung

```
A→B  richtig (A eindeutig)
B→D  richtig (x→5,5 · y→6,6)
C→B  falsch  (p→x und p→y)
BC→A richtig (alle BC-Paare verschieden)
D→B  richtig (5→x,x · 6→y,y)
```

E6 – Text → min-max → Teorey → Relation

„Ein Kunde kann beliebig viele Bestellungen aufgeben. Jede Bestellung gehört zu genau einem Kunden.“

▼ Lösung

```
min-max:  Kunde (0,*) — gibt auf — (1,1) Bestellung
Teorey:   Kunde —|— ◇◆ —o— Bestellung   (gefüllt zu Bestellung)
Relation: Kunde(KNr, …)   Bestellung(BNr, …, #KNr NN)
```

E7 – kleines EER → Relationen

Schule: Lehrer(LNr, Name), Klasse(Bez, Jahrgang). Jede Klasse hat genau einen Klassenvorstand, ein Lehrer ist höchstens einer Klasse KV. Lehrer unterrichten viele Klassen, Klassen haben viele Lehrer; gespeichert wird die Stundenanzahl. Ein Raum ist pro Gebäude(GNr) durch seine RNr eindeutig; jede Klasse hat genau einen Stammraum.

▼ Lösung

```
Lehrer       (LNr, Name)
Gebaeude     (GNr)
Raum         (#GNr, RNr)                        -- Weak Entity
Klasse       (Bez, Jahrgang, #KV_LNr NN UNIQUE, #GNr NN, #RNr NN)
unterrichtet (#LNr, #Bez, Stunden)              -- n:m
```

KV: 1:1, FK auf die Seite, wo genau 1 (Klasse), UNIQUE, weil ein Lehrer max. einmal KV ist.

E8 – DDL

Erstelle Kurs(KursNr, Titel, Preis ≥ 0, Level ∈ {A1,A2,B1}) und Anmeldung(KursNr, TeilnehmerNr, Datum = heute). Beim Löschen eines Kurses sollen Anmeldungen mitgelöscht werden. Danach: Spalte *MaxPlaetze* (Ganzzahl > 0) zu Kurs hinzufügen und Titel auf 200 Zeichen erweitern.

▼ Lösung

```
CREATE TABLE Kurs (
  KursNr INT PRIMARY KEY,
  Titel  VARCHAR(100) NOT NULL,
  Preis  DECIMAL(8,2) CHECK (Preis >= 0),
  Level  VARCHAR(2) CHECK (Level IN ('A1','A2','B1'))
);
CREATE TABLE Anmeldung (
  KursNr       INT,
  TeilnehmerNr INT,
  Datum        DATE DEFAULT (CURRENT_DATE),
  PRIMARY KEY (KursNr, TeilnehmerNr),
  FOREIGN KEY (KursNr) REFERENCES Kurs(KursNr) ON DELETE CASCADE
);
ALTER TABLE Kurs ADD MaxPlaetze INT CHECK (MaxPlaetze > 0);
ALTER TABLE Kurs MODIFY Titel VARCHAR(200) NOT NULL;
```

E9 – ternäre Beziehung

Lieferant – Teil – Projekt, Beziehung *liefert*. Für ein Projekt und ein Teil gibt es genau **einen** Lieferanten (Lieferant = 1-Seite). Schlüssel?

▼ Lösung

```
Projekt, Teil → Lieferant
liefert(#ProjektNr, #TeilNr, #LieferantNr)
```

---

10 · Spickzettel

Notation

min-max beim **eigenen** Entity
Teorey/Chen **über Kreuz**
| ⇒ min 1 · o ⇒ min 0 · gefüllt ⇒ max *

EER → Relation

1:1 → FK eine Seite

1:n → FK auf n-Seite

n:m / ternär → eigene Relation

Weak → Besitzer-PK + Teilschlüssel

IS-A → gleicher PK

Schlüssel

nie rechts ⇒ in jedem Schlüssel

nur rechts ⇒ in keinem

$X^+ = R$ ⇒ Superschlüssel, minimal ⇒ Schlüssel

Normalformen

2NF ×: Teil des Schlüssels → nichtprim

3NF ×: Nicht-Schlüssel → nichtprim

Superschlüssel links ⇒ immer ok

alle prim ⇒ 3NF

Synthese

F_c ⇒ pro FD eine Relation ⇒ Schlüssel-Relation falls nötig ⇒ Teilmengen streichen

SQL

CHECK (x IN ('a','b')) · DECIMAL(10,2)

FOREIGN KEY (a) REFERENCES T(a) ON DELETE CASCADE

ALTER TABLE t ADD / DROP COLUMN / MODIFY

Quellen: insy4_01_EER, insy4_02_Umsetzung, insy4_03_Normalformen, insy_03b_Synthese, Übungen 1–4.1 inkl. Lehrer-Lösungen, Tests 24.10.2024 und 22.10.2025.